

Universidad CENFOTEC
Escuela de Software

Primer estudio de caso

Aplicación de las pilas al análisis de cadenas de impresión

Curso: SOFT-10, Estructuras de Datos

Sección: SCV5 | Periodo: C3-2026

Docente facilitador: Romario Salas Cerdas

Estudiante: Braden Lee Knuter Cordoba

Fecha de entrega: 11 de octubre del 2026

Repositorio del proyecto en GitHub:

<https://github.com/B0W3YYY/CasoDeEstudio1-EstructuraDeDatos>

1. Alternativa seleccionada

De las dos aplicaciones propuestas en la consigna, este trabajo desarrolla el análisis de cadenas de impresión. El programa recibe una expresión como las que se escriben dentro de una instrucción de impresión y determina si está bien formada. Los tokens que contempla son los tres que pide el enunciado: cadenas literales delimitadas por comillas dobles, variables de cadena identificadas por su nombre y el operador de concatenación +. A ellos se agregan los paréntesis de agrupación, que obligan a recordar un contexto abierto y son los que hacen necesaria la pila.

Una expresión válida es "Hola, " + nombre + ("!" + salto). Una inválida es ("Total: " + monto, porque el paréntesis que se abrió nunca se cerró. El problema consiste en decidir si cada delimitador de apertura encuentra su cierre y si los operandos y los operadores se alternan como corresponde. La pila se programó en Java con nodos enlazados, sin usar las clases de colección del lenguaje.

2. Por qué una pila es la estructura de datos adecuada

El requerimiento central del análisis de cadenas de impresión es verificar el correcto anidamiento de los delimitadores. Esa verificación tiene una característica que determina la estructura de datos: cuando aparece un delimitador de cierre, el único delimitador de apertura con el que puede emparejarse es el más reciente que siga pendiente. En la expresión ("a" + (b + "c")), el primer paréntesis de cierre no corresponde al paréntesis inicial sino al segundo, porque es el último que quedó abierto. Ese comportamiento, donde el último elemento que entra es el primero que sale, es la política LIFO (*Last In, First Out*) que define a una pila.

La correspondencia entre el problema y la estructura es formal. El lenguaje de los paréntesis balanceados no es un lenguaje regular, de modo que no puede reconocerse con un autómata de estados finitos: la profundidad de anidamiento es ilimitada y un autómata finito no puede llevar una cuenta arbitraria. Pertenece a los lenguajes independientes del contexto, y el modelo de máquina que reconoce esa clase es el autómata de pila (Aho, Lam, Sethi y Ullman, 2007). La pila es entonces la memoria mínima que el problema exige.

Las alternativas quedan descartadas por razones concretas. Un contador de paréntesis abiertos detectaría que faltan cierres, pero no podría distinguir entre

comillas y paréntesis ni informar en qué posición quedó abierto el delimitador que nunca se cerró. Una lista o un arreglo recorridos libremente resolverían el problema, aunque ofrecerían accesos que el algoritmo no necesita y que, usados por error, producirían emparejamientos incorrectos. Una cola, con su política FIFO, emparejaría cada cierre con el delimitador más antiguo, que es lo contrario de lo que pide el anidamiento. La pila expone solo las tres operaciones que el problema requiere, y esa restricción es parte de lo que vuelve correcto al algoritmo.

3. Orden, seguridad y eficiencia

3.1 Orden

La pila impone el orden de resolución sin que el programador tenga que administrarlo. Al recorrer la expresión de izquierda a derecha, cada apertura se apila y cada cierre consume la cima, por lo que los delimitadores se resuelven en orden inverso al de su aparición, que es el orden correcto del anidamiento. El estado de la pila en cualquier momento describe el contexto en el que se está leyendo: su tamaño es la profundidad actual de anidamiento y su contenido, de la cima a la base, es la lista de contextos abiertos. Esa información permite emitir un diagnóstico preciso en lugar de un aviso genérico de error.

3.2 Seguridad

La seguridad que aporta la pila opera en dos niveles. En el del algoritmo, la interfaz reducida de la estructura vuelve imposibles ciertos errores: como no existe un método para leer o modificar un elemento intermedio, no hay forma de emparejar un cierre con un delimitador que no sea el correcto. En el del programa, la pila de este proyecto está diseñada para no interrumpir la ejecución ante un uso indebido. Los métodos `pop()` y `peek()` devuelven `null` cuando la pila está vacía en lugar de lanzar una excepción, de modo que quien los invoca comprueba el resultado y muestra un mensaje al usuario. Por eso una entrada mal formada como "Hola" + nombre) produce una explicación y no una terminación abrupta.

El análisis de delimitadores es además una forma de validación de entrada. Revisar la estructura de una cadena antes de procesarla es una práctica de seguridad conocida, porque una cadena con comillas sin cerrar puede alterar el significado del texto que la sigue, problema emparentado con las inyecciones de código que

aparecen al construir instrucciones concatenando texto sin verificarlo.

3.3 Eficiencia

Las tres operaciones de la pila trabajan únicamente sobre la cima y no recorren la estructura, así que su costo es constante, $O(1)$. El analizador ejecuta un número acotado de operaciones por cada carácter y visita cada carácter una sola vez, de manera que el análisis completo de una expresión de longitud n se resuelve en tiempo $O(n)$, en una sola pasada. La memoria utilizada no depende de la longitud de la expresión sino de su profundidad máxima de anidamiento, que en una cadena de impresión real es muy pequeña, porque solo se almacenan los delimitadores que permanecen abiertos. Al estar construida con nodos enlazados, la pila crece y decrece sin reservar espacio por anticipado ni copiar elementos.

4. Ejemplo concreto en software de sistema

El caso más directo de esta funcionalidad en software de sistema es el compilador de Java, `javac`, incluido en el JDK. Antes de traducir un programa, el compilador divide el texto fuente en tokens y verifica que estén bien formados, y entre esas comprobaciones están las dos que realiza el analizador de este proyecto: que toda cadena literal se cierre y que los paréntesis de una instrucción de impresión estén balanceados. La especificación del lenguaje establece en su sección 3.10.5 que un literal de cadena está delimitado por comillas dobles y que constituye un error de compilación que aparezca un terminador de línea entre la comilla de apertura y la de cierre, lo que vuelve detectable una comilla sin cerrar (Oracle, 2023). El análisis sintáctico que resuelve el anidamiento de paréntesis se apoya en un autómata de pila (Aho et al., 2007).

Para ilustrar el comportamiento se compiló un archivo que contiene los dos errores de forma deliberada. La figura 1 muestra la salida de `javac`:

```
●●● Compilador javac (JDK 26) - deteccion de delimitadores sin cerrar

$ javac Demo.java
Demo.java:4: error: unclosed string literal
    System.out.println("Hola, " + nombre + "!);
                                   ^
Demo.java:5: error: ')' or ',' expected
    System.out.println(("Total: " + nombre);
                                   ^
2 errors
```

Figura 1. Salida del compilador javac del JDK 26 al procesar un archivo con una cadena literal sin cerrar y un paréntesis sin cerrar. El compilador señala la línea y marca con el símbolo ^ la posición exacta del problema.

El compilador no se limita a indicar que el archivo es incorrecto. Identifica el tipo de error (*unclosed string literal*, *)' expected*) y la posición donde ocurre. Esa precisión es posible porque conserva los contextos abiertos, de modo que al terminar de leer sabe cuáles quedaron pendientes. El programa desarrollado en este estudio de caso reproduce el mismo comportamiento sobre las dos expresiones equivalentes, como se observa en la figura 2, donde además se muestra el estado de la pila después de cada operación.

●●● AnalizadorCadenas.java - mismo diagnostico con la pila propia

```
$ java -cp out Main
Analizando: "Hola, " + nombre + "!
  push '"' (posicion 0) -> [cima] " [base]
  pop '"' (posicion 7) -> (pila vacia)
  push '"' (posicion 20) -> [cima] " [base]
ERROR en la posicion 20: la comilla de apertura no fue cerrada.
Analizando: ("Total: " + nombre
  push '(' (posicion 0) -> [cima] ( [base]
  push '"' (posicion 1) -> [cima] " ( [base]
  pop '"' (posicion 9) -> [cima] ( [base]
ERROR en la posicion 0: el delimitador '(' no fue cerrado.
Programa finalizado. Hasta pronto.
```

Figura 2. Salida del analizador desarrollado (opción 2 del menú) sobre los mismos dos casos defectuosos. La traza muestra cada push y cada pop junto con el contenido de la pila, y el diagnóstico final indica la posición del delimitador sin cerrar.

Al comparar ambas figuras se observa que la lógica es la misma: el delimitador que se abre se recuerda, y la falta de su cierre se detecta porque al terminar el recorrido sigue pendiente. El mismo mecanismo aparece en los editores de código, que resaltan el paréntesis correspondiente al que tiene el cursor, y en los intérpretes interactivos, que mantienen abierta la entrada mientras quede un delimitador sin cerrar.

5. Representación gráfica de las operaciones

Las figuras siguientes representan la pila como una columna de casillas. Cada casilla corresponde a un nodo de la clase Pila y la casilla superior ocupada es la cima. La figura 3 muestra las operaciones fundamentales sobre la pila de delimitadores.

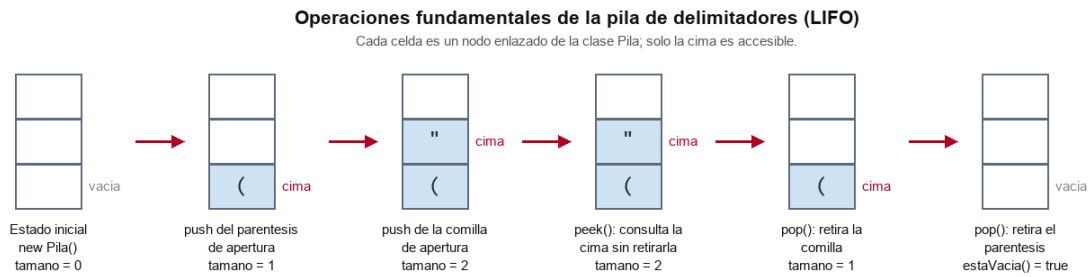


Figura 3. Operaciones fundamentales de la pila: estado inicial, dos operaciones push, una consulta peek que no modifica la estructura y dos operaciones pop que la devuelven al estado vacío.

La figura 4 presenta la traza completa del análisis de una expresión válida. Los pasos y las posiciones indicadas coinciden con la salida que produce la opción 2 del menú.

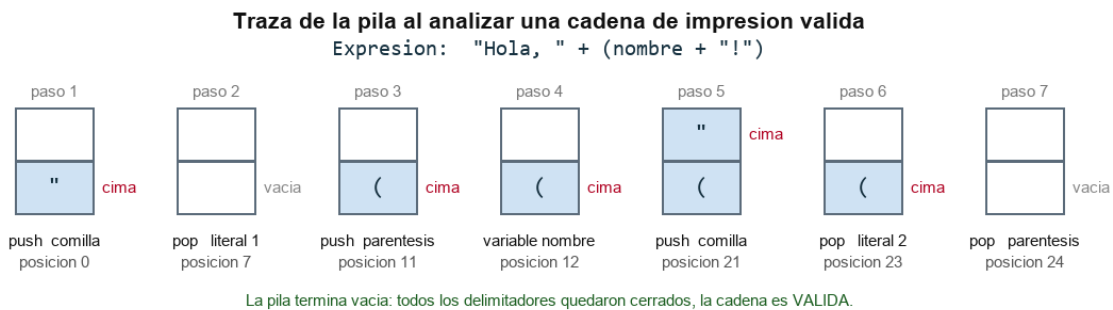
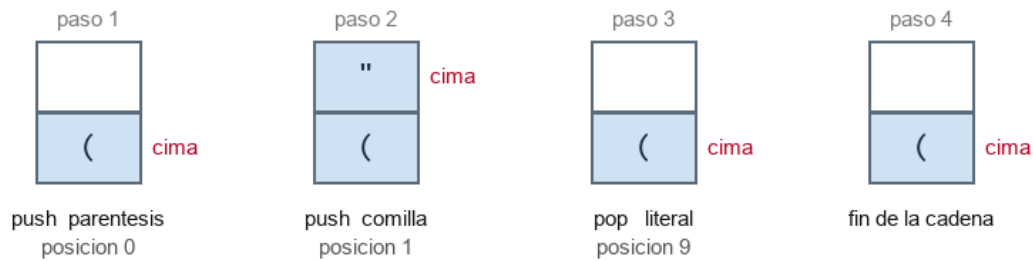


Figura 4. Traza del análisis de la expresión "Hola, " + (nombre + "!"). La pila termina vacía, lo que confirma que todos los delimitadores fueron cerrados.

La figura 5 corresponde al caso inválido. La pila no queda vacía al terminar el recorrido, y esa condición permite informar cuál delimitador quedó abierto y en qué posición.

Deteccion de un delimitador sin cerrar

Expresion: ("Total: " + nombre



peek() informa que el '(' abierto en la posicion 0 nunca se cerro: la cadena es INVALIDA.

Figura 5. Análisis de la expresión inválida ("Total: " + nombre. Al finalizar, el paréntesis de la posición 0 continúa en la pila, por lo que la cadena se rechaza.

La figura 6 resume el flujo de control del método que realiza el análisis e incluye las operaciones que la funcionalidad necesita además de las fundamentales: la clasificación de cada carácter y la verificación de que operandos y operadores se alternen correctamente.

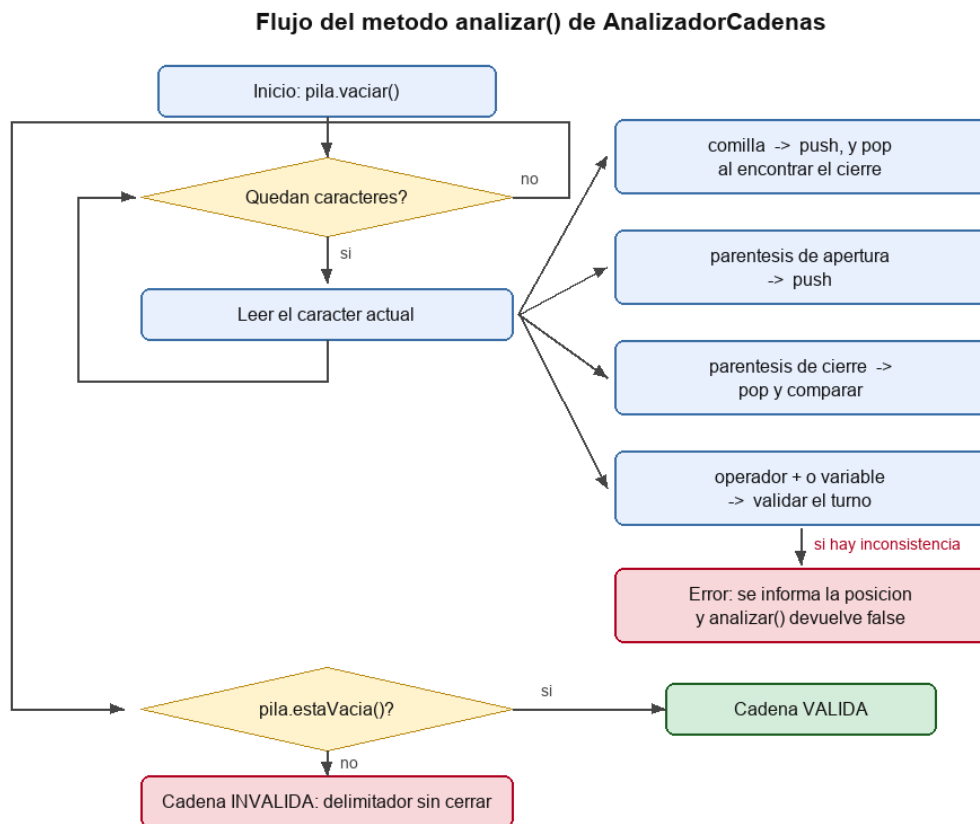


Figura 6. Flujo del método analizar(): recorrido carácter por carácter, clasificación del token, operaciones sobre la pila y verificación final del estado de la pila.

6. Clases del programa

El proyecto está formado por cuatro clases. El código fuente completo se encuentra en el repositorio indicado en la portada y en la sección 8.

- **Nodo** representa un elemento de la pila. Tal como indica la consigna para la alternativa seleccionada, el dato almacenado es un carácter que forma parte de la cadena por analizar. Guarda también la posición que ese carácter ocupa dentro de la expresión, dato que permite informar el lugar exacto de un delimitador sin cerrar, y una referencia al nodo situado debajo.
- **Pila** implementa la estructura con nodos enlazados. Ofrece push, pop, peek, estaVacia, getTamano, vaciar, contenido y mostrar. Las dos últimas permiten visualizar el estado de la estructura.
- **AnalizadorCadenas** contiene la lógica del análisis en el método analizar, que recorre la expresión una sola vez, usa la pila para los delimitadores, cuenta

literales, variables y operadores, y puede emitir la traza paso a paso.

- **Main** es el entorno de ejecución. Su método `main()` invoca a `menu()`, que ofrece cinco opciones: analizar una cadena, analizarla mostrando la traza de la pila, ejecutar un conjunto de ejemplos, operar la pila manualmente mediante un submenú de push, pop y peek, y salir. Las entradas no numéricas o fuera de rango se rechazan con un mensaje y no interrumpen el programa.

7. Conclusiones y recomendaciones

El análisis de cadenas de impresión muestra que la elección de la estructura de datos determina si el algoritmo puede ser correcto. El anidamiento de delimitadores exige recordar contextos y resolverlos en orden inverso al de su apertura, y la pila realiza esa política con costo constante por operación. La implementación resultante recorre la entrada una sola vez, usa memoria proporcional a la profundidad de anidamiento y produce diagnósticos precisos, igual que un compilador real.

Para una extensión futura del trabajo se recomienda admitir secuencias de escape dentro de las cadenas literales, de modo que una comilla precedida por una barra invertida no se interprete como cierre. También sería útil reportar todos los errores de una expresión en lugar de detenerse en el primero, mediante una recuperación parecida a la que emplean los compiladores.

8. Repositorio del proyecto

El código fuente completo y ejecutable del proyecto de Java está disponible en el siguiente repositorio público de GitHub:

<https://github.com/B0W3YYY/CasoDeEstudio1-EstructuraDeDatos>

El repositorio contiene la carpeta `src` con los archivos `Nodo.java`, `Pila.java`, `AnalizadorCadenas.java` y `Main.java`, junto con un archivo `README.md` que explica cómo compilar y ejecutar el programa.

9. Referencias

Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2007). *Compilers: Principles, Techniques, and Tools* (2.^a ed.). Pearson/Addison-Wesley.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4.^a ed.). The MIT Press.

Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). *Data Structures and Algorithms in Java* (6.^a ed.). John Wiley & Sons.

Oracle. (2023). *The Java Language Specification, Java SE 21 Edition*. Sección 3.10.5, String Literals.
<https://docs.oracle.com/javase/specs/jls/se21/html/jls-3.html>

Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4.^a ed.). Addison-Wesley.